

Assignment on Binary Creation

Problem 1 (JIT Compilation)

In the lectures, we have looked at two models of execution: The compilation model and the interpretation model. In this assignment, we shall look at a third model which has features of both. This is called **Just-in-Time (JIT) compilation**. It starts with the interpretation model, but on discovering paths (typically in loops) which are "hot" in the sense that they are being executed very frequently, the paths are optimized and converted into machine code. We shall specifically look at the JIT compilation of Python; the compiler is called **pypy**.

The result is that we do not lose the flexibility and ease of the interpretation model, especially for dynamically typed languages. We also bring in the efficiency of compilation for frequently executing paths. So, this is a compromise between the earlier two models.

Note that after stating the problem, I will give you very broad hints and you have to find out the answers on your own. You may consult an LLM, you may look at references, you can look at the **pypy** manuals, and so on. In general, you have to learn by discovering things on your own.

Here is a summary of JIT compilation of Python:

1. In Python, source code is first compiled into bytecode—a low-level, platform-independent representation executed by the **pypy** bytecode interpreter. To familiarize yourself with this, define a simple function `add(a,b)` that returns their sum. Then, `import dis` (disassembler) and do `dis.dis(add)`. Observe how the interpreter utilizes the stack to handle variables and operations.
2. The interpreter runtime keeps track of frequently executed paths, especially loops.
3. When a path becomes "hot" (based on predetermined criteria, such as an iteration threshold), the JIT begins **tracing lower-level operations arising from executing the bytecodes in that path**. The trace records the **specific operations performed during that execution** (note this carefully).
4. The recorded trace is then optimized to remove interpreter overhead. Type-based assumptions are captured as guards to ensure that the specialized machine code remains valid for subsequent executions. **pypy**'s optimizer works on traces, which are linear sequences of instructions with guards. Here is an example. First consider the bytecode sequence.

```
LOAD_FAST 0 // take a from frame and push it on stack
LOAD_FAST 1 // take b from frame and push it on stack
BINARY_ADD // Perform type specific addition
```

To execute these three instructions, the interpreter performs a sequence of actions roughly like this:

```
fetch LOAD_FAST 0 // read next bytecode instruction, LOAD_FAST 0
push 0           // place value of a on the operand stack
fetch LOAD_FAST 1 // read next bytecode instruction, LOAD_FAST 1
push 1           // place value of b on the operand stack
fetch BINARY_ADD  // read next bytecode instruction, BINARY_ADD
```

```
check type(0), type(1) // inspect runtime types of a and b
integer_add 0, 1 // perform integer addition if both are integers
```

Now once the JIT compiler has the above record of actions, it can optimize it to the following sequence:

```
load 0 // From interpreter stack, to a register if
available, else hardware stack
guard: value is integer // Code is valid only if guard is true
load 1
guard: value is an integer
integer add
```

These instruction sequences are oversimplified to keep explanations simple. In your assignment, you have to find the actual bytecode instructions the optimized interpreter trace and even the disassembled machine code.

6. The optimized trace is converted into machine code for the current processor. This machine code is stored in executable memory (between stack and heap) inside the running process.
7. On later visits, execution jumps to the compiled code, whose guards validate the assumptions during execution.
8. If a guard fails, control exits the compiled code. pypy must rebuild the interpreter-visible state before resuming at an appropriate continuation point in the bytecode.

End of story.

Now consider the following program with three loops, out of which two will eventually become hot:

```
import time
import __pypy__

def two_loops_series(n):
    total1 = 0
    for i in range(n): // This is Loop 2, a hotspot
        total1 += i

    mid = total1 * 2
    offset = mid + 7
    temp = offset - total1

    total2 = 0
    for j in range(n): // This is Loop 3, a hotspot
        total2 += j * temp

    return total1, total2, temp

if __name__ == "__main__":
    for _ in range(5): // This loop is not a hot spot
        result = two_loops_series(300)
```

```
end = time.time()

print(f"Result: {result}")
```

We would like to answer the following questions with respect to the pypy jit interpreter.

1. What is the bytecode sequence for `total1 += i`? Hint: Use `dis`. [2 points]

2. At which iteration numbers are **Loop 1** and **Loop 2** detected to be hotspots? Hint: Instrument the code to set up counters to count the number of iteration of the two loops.
 - `import pypyjit`. It provides `pypyjit.set_compile_hook(callback, False)`, which invokes `callback` whenever pypy compiles a JIT loop. The callback should record the current values of the `loop1` and `loop2` counters.
 - The function `callback` receives a `pypyjit.JitLoopInfo` object. [2 points]
3. What is the optimized interpreter trace sequence for `total1 += i` inside the first loop. Note that being in the loop means that the statement could be executed more than once. Hint:
 - Generate `my_trace.log` using `PYPYLOG=jit-log-opt:my_trace.log pypy3 <program.py>`.
 - In `my_trace.log`, first locate the loop corresponding to **Loop 2**. You can identify it from the function name and the bytecode offset of the first `FOR_ITER`.
 - Inside that loop, search for the `debug_merge_point(...)` entries corresponding to the bytecodes of `total1 += i`.
 - Around those markers, identify the trace instructions that implement the optimized version of the statement. Indicate how the updated value of `i` and `total1` are carried forward to the next iteration. [(2+2) points]
4. What is the disassembled machine code for `total1 += i`? Hint:
 - Generate the backend dump using `PYPYLOG=jit-backend:dump.asm pypy3 <program.py>`.
 - In `dump.asm` search for the string **Loop 2**. Under this string the starting address of different sections are recorded. Our interest is in the range of addresses [`resops`, `failures`). "Resops", short for "Residual Operations", are the operations that remain after the JIT has optimized away all the slow Python overhead.
 - Find the `CODE_DUMP` blob that starts with the address of `gc_table`. In this blob, extract the bytes in the range: [`resops`, `failures`) in a file `<file.bin>`.
Note: Be very careful, A single byte here or there will produce garbage in the next steps.
 - Convert the hex dump into raw bytes using `xxd -r -p`. Record the result in `<file_binary.bin>`.
 - Disassemble it using a tool such as `objdump`.
`objdump -D -b binary -m i386:x86-64 --adjust-vma=<resops>`

Assuming `resops` is `000076648c17f38b` we shall get:

`000076648c17f38b <.data>:`

```
76648c17f38b: 49 ff 03                incq    (%r11)
76648c17f38e: 48 8b 8d 28 01 00 00 mov     0x128(%rbp),%rcx
76648c17f395: 48 8b 41 28             mov     0x28(%rcx),%rax
76648c17f399: 48 8b 51 40             mov     0x40(%rcx),%rdx
...
```

- In the resulting assembly, identify the instructions corresponding to:
 - the end-of-loop test, [2 points]
 - the code for `total1 += i`, [2 points]
 - any overflow checks, [2 points]
 - and the backedge jump. [2 points]

[Q1 Total: 16 points]

Problem 2 (Make and CMake)

Create a Makefile named `rawmake` (can be run using `make -f rawmake`) and fill it up as instructed in the tasks below. Source and Header files are provided in `resources/q2` directory. Assume that `rawmake` file will be placed directly under `q2/` directory. The dir structure after creating `rawmake` will be as follows

```
q2/
├── helloworld.cpp
├── myengine
│   ├── myengine.cpp
│   └── myengine.hpp
├── mygame
│   └── mygame.cpp
├── rawmake
└── usespthreads.cpp
```

1. Add a rule to compile `helloworld.cpp` to form `helloworld` executable. The build command should be: `make -f rawmake helloworld` [1 point]
2. Add a rule to compile `usespthreads.cpp` to form the `usespthreads` executable. This file uses the `pthread` library used to create and manage threads. In order to properly compile this file you need to special use a flag for `g++`. The build command should be `make -f rawmake usespthreads` [2 points]
3. In this task we build a library using files `myengine.hpp` and `myengine.cpp`. Use the given resources to add rules to make dynamic and static libraries. Dynamic library created should have name `libMyEngineDynamic.so` and similarly static `libMyEngineStatic.a`. The build commands should be `make -f rawmake libMyEngineDynamic.so` and `make -f rawmake libMyEngineStatic.a` respectively. [2+2 points]
4. Now we use the installed libraries in `mygame.cpp`. Add a rule to compile `mygame.cpp` with the static library installed in the previous task and produce binary name `mygamestatic`. Build command should be `make -f rawmake mygamestatic` [2 points]
5. Add a rule to compile `mygame.cpp` with the dynamic library installed in the previous task and produce binary name `mygamedynamic`. Build command: `make -f rawmake mygamedynamic` [2 points]
6. Also add a `PHONY` rule to clean all the generated intermediates and binaries (`.o .a .so` and other binaries). The build command should be `make -f rawmake clean` [1 point]
7. Now your job is to do the same tasks using CMake. To do this you need to create a `CMakeLists.txt` file (directly under `q2/` directory) which generated Makefile to build and install targets in the same manner as above tasks 1 - 4
 1. `helloworld` binary from `helloworld.cpp` [1 point]
 2. `usespthreads` binary from `usespthreads.cpp` [2 points]
 3. `libMyEngineDynamic.so` and `libMyEngineStatic.a` libraries from `myengine.hpp` and `myengine.cpp` [2 + 2 points]

4. Instead of installing two types of libraries separately, it should now install both of the libraries and the corresponding header file **myengine.hpp** to **/usr/local/lib/** and corresponding headers to **/usr/local/include/** respectively. Build command:
- ```
sudo make install
```
- [2 points]

Observe that the makefile that **CMakeLists.txt** creates already contains PHONY rule **clean** which removes all the targets and intermediates generated

You should build the project using the following commands:

```
mkdir build
cd build
cmake ...
make # (should create helloworld, usespthread,
libMyEngineStatic.a, libMyEngineDynamic.so)
sudo make install # (should install libMyEngineStatic.a,
libMyEngineDynamic.so and header into specified paths
appropriately)
```

8. Now we want to do tasks 4 and 5 using **CMake**. To do this create a **CMakeLists.txt** file in **q2/mygame** directory. This file should compile **mygame.cpp** with the static library installed in the previous task and produce binary name **mygamestatic** and similarly compile **mygame.cpp** with the dynamic library installed in the previous task and produce binary name **mygamedynamic**
- [2 + 2 points]

Command run while checking: (from mygame dir)

```
mkdir build
cd build
cmake ..
make # should create mygamestatic and mygamedynamic
```

The directory structure after creating the **rawmake** file and two **CMakeLists.txt** files will be as follows

```
q2/
├── CMakeLists.txt (For task 7)
├── helloworld.cpp
├── myengine
│ ├── myengine.cpp
│ └── myengine.hpp
├── mygame
│ ├── CMakeLists.txt (For task 8)
│ └── mygame.cpp
├── rawmake (For tasks 1-6)
└── usespthread.cpp
```

### Problem 3 (Linking)

In this assignment, you will trace, as concretely as possible, how a call to `printf` in the example program

```
#include <stdio.h>
int main() {
 printf("Hi\n");
 return 0;
}
```

is transformed into a jump to its final runtime address in the library `libc.so`. You are not being asked to describe the mechanism in words. Instead, you must recover the actual intermediate artifacts involved in this process, such as the call instruction, the PLT entry, the GOT entry, the relevant relocation record, the dynamic symbol-table entry, and the final value written into `GOT[printf]`. Your goal is to follow the chain from the source-level call all the way to the moment when `GOT[printf]` is updated with the resolved runtime address. Initially, this GOT entry **does not** contain the final address of `printf`.

Before starting, note the following carefully. ELF sections primarily used for static linking, such as `.symtab`, `.rela.text/.rel.text`, and `.rela.data/.rel.data`, do not play a role *during execution* and are therefore not loaded into memory. In contrast, dynamic linking relies on sections such as `.plt`, `.got.plt`, `.dynamic`, `.dynsym`, and `.rela.plt` *during execution*. So each of these tables has a counterpart in memory.

Notation: In the following questions, when I say `PLT[printf]`, I mean the address of the PLT entry for `printf`. Similarly, `GOT[printf]` means the address of the GOT entry for `printf`. When I say `*GOT[printf]`, I mean content of the GOT entry for `printf`. Thus 'pushq `*GOT[1]`' means "push the content of `GOT[1]` onto the stack". Warning: This is not the usual notation in which the instruction decides whether to dereference the address or not.

The process to be traced in this assignment is the following:

1. Compile the program with `gcc -g -no-pie -fno-builtin-printf -o hello hello.c`. Think why we used each of these switches: `-g`, `-no-pie`, `-fno-builtin-printf`. Now let us do some excavation to confirm whatever you have learnt in the slides.
2. What is the address of the PLT entry for `printf`? Or, in our notation, what is `PLT[printf]`? (use: `objdump -d`)

3. According to the slides, the first instruction of `PLT[printf]` should be an indirect jump through the GOT entry for `printf`, that is, `jump *GOT[printf]`. So what are `GOT[printf]` and `*GOT[printf]`? (use: `objdump -d`, `objdump -s -j .got.plt`)
4. So we are back to PLT once again. Now the instructions after `PLT[printf]` should be a `push $index` and a `jump PLT[0]`. So what are `$index` and `PLT[0]`? (use: `objdump -d`, `objdump -s -j .got.plt`)
5. The instructions at `PLT[0]` should be `push *GOT[1]` and `jump *GOT[2]`. What are `*GOT[1]` and `*GOT[2]`? (use: `objdump -d`, `objdump -s -j .got.plt`)
6. Now assume that at this point execution has started. Any further excavation will involve `gdb`. So, run `gdb` on the executable and set a breakpoint just before the call to `printf`. Note the values of `*GOT[1]`, `*GOT[2]` and `*GOT[printf]`. Step over the `printf` and once again note the value of `*GOT[printf]` again.

Now fill the table below with your observations. For filling the blue portion of the table use `objdump`, for the red portion, use `gdb`.

(Useful: `gdb`, `break`, `run`, `continue`, `x`, `si`, `ni`)

| S.No | Step | Context                                                  | Value required            | Value |
|------|------|----------------------------------------------------------|---------------------------|-------|
| 1    | 2    | <code>call PLT[printf]</code>                            | <code>PLT[printf]</code>  |       |
| 2    | 3    | <code>jump *GOT[printf]</code>                           | <code>GOT[printf]</code>  |       |
| 3    | 3    | <code>jump *GOT[printf]</code>                           | <code>*GOT[printf]</code> |       |
| 4    | 4    | <code>push \$index</code>                                | <code>\$index</code>      |       |
| 5    | 4    | <code>jmp PLT[0]</code>                                  | <code>PLT[0]</code>       |       |
| 6    | 5    | <code>push *GOT[1]</code>                                | <code>GOT[1]</code>       |       |
| 7    | 5    | <code>push *GOT[1]</code>                                | <code>*GOT[1]</code>      |       |
| 8    | 5    | <code>jump *GOT[2]</code>                                | <code>GOT[2]</code>       |       |
| 9    | 5    | <code>jump *GOT[2]</code>                                | <code>*GOT[2]</code>      |       |
| 10   | 6    | Just before call to <code>printf</code>                  | <code>*GOT[1]</code>      |       |
| 11   | 6    | Just before call to <code>printf</code>                  | <code>*GOT[2]</code>      |       |
| 12   | 6    | just before the call to <code>printf</code>              | <code>*GOT[printf]</code> |       |
| 13   | 6    | just after stepping over the call to <code>printf</code> | <code>*GOT[printf]</code> |       |

[13 x 2 = 26 Points]

You can see that there is a jump in our excavation process. We have assumed that the link-map address, the resolver and index are enough to resolve the address of `printf` and update the `GOT[printf]` entry. But we have not actually seen the process of resolution and updating. We shall try do it as an exercise later in the course.

Happy excavating 😊